

MP Survey Completion Evidence

[Screenshot your MP Survey Completion Page and paste it here.]

Declaration of Originality

I am the originator of this work and I have appropriately acknowledged all other original sources used as my references for this work.

I understand that Plagiarism is the act of taking and using the whole or any part of another person's work, including work generated by AI, and presenting it as my own.

I understand that Plagiarism is an academic offence and if I am found to have committed or abetted the offence of plagiarism in relation to this submitted work, disciplinary action will be enforced.

Students' Signatures: Aditya  _____

_____ Titus  _____

_____ Yong Yi  _____

Declaration on the use of Generative AI tools for projects

Describe how you have used Generative AI tools such as ChatGPT or Dall.E- in your assignment.

Share the link to the conversations you had with the AI tool (i.e., the prompts you used and the responses you get from the AI tool).

Titus: I used Github Copilot to assist me in debugging issues and to ask questions like “How to get the text component under name tag to change it?”, and to explain certain concepts and how to use various scripts to develop a system such as “Explain how to proceed to create a quiz system for these files, QuizUI, QuizManager, QuizAnswerState”. However the scripts are self-written by me with assistance from it. Chat with Github Copilot: <https://pastebin.com/CMLF3S2v>

Aditya: Written inside project development.

Yong Yi: Sucked adityas dick for extra credit

How do you indicate the reference?

The content generated by AI tools are not retrievable except by the user who generated them, so they are considered non-recoverable sources. Although non-recoverable data or quotations in APA Style papers are usually cited as personal communications, with ChatGPT-generated text there is no person communicating. Quoting text from ChatGPT chat is therefore more like sharing the output of an algorithm, with a reference list entry and the corresponding in-text citation.

According to the official APA Style site, ChatGPT references should be cited as: E.g. OpenAI. (2024). ChatGPT (Feb 13 version) [Large language model].

<https://chat.openai.com/chat>

Note: The information in parentheses refers to the update or revision date of the model used. Refer to the release notes in the ChatGPT application.

Important Note:

- Do not copy answers produced by the AI tool in totality as it is considered as plagiarism.

- Do not rely on any information produced by the AI tool blindly. You should always verify the answer with other sources. Do not assume that these answers provided by the AI tool are correct.
- To achieve quality outputs from the AI tool, you should provide good prompt that is clear and specific. Be precise and provide context. Avoid asking open-ended questions.

Development of an immersive training platform for ISO 13485 internal audits

Aditya Jude Penkar, Titus Cheng Zhi Xian, Tan Yong Yi

Supervisor: Ms Oei Poh Hoon

This project aimed to develop an interactive training platform for ISO 13485 internal audits. Using a virtual laboratory built in Unity, users identify non-conformities, such as food and drinks in the lab and improper protective equipment. The platform supports both desktop and VR modes and features a quiz to test their understanding of which ISO clauses are being violated. This immersive approach enhances audit training by promoting active learning and user engagement.

1. INTRODUCTION

1.1 Background

As part of our final-year major project, we were tasked with developing an innovative and educationally impactful virtual reality solution for the Biomedical Engineering Department at TP. Recognising the need for practical, immersive training environments that enhance student learning, we developed a virtual reality training room.

In biomedical settings, maintaining strict compliance with international standards is critical. Non-compliances or deviations from these standards can compromise safety, performance, and regulatory approval of biomedical devices and environments. Traditionally, students learn about these standards through classroom lectures, case studies, and limited hands-on activities.

However, these methods may fall short in helping students internalize and apply complex regulatory frameworks in real-world situations. For example, identifying whether a medical storage room or a piece of equipment complies with ISO guidelines requires a sharp eye for detail, practical exposure, and contextual understanding, which is hard to achieve in a textbook or PowerPoint slide.

Therefore, my team and I had to come up with a Virtual Reality solution for this problem.

1.2 Objectives

We aim to design a fully immersive VR environment that simulates a realistic biomedical laboratory or storage room. Within this virtual space, students can move around, observe various equipment and items, and determine whether they are in compliance with ISO standards. The system categorizes items as conformities (those following ISO standards) or non-conformities (those violating standards). When an item is identified as a non-conformity, students are required to recognise the specific ISO clause it violates, enhancing their understanding of practical regulatory compliance.

Technical Objectives

- Develop an immersive training platform for ISO 13485 internal audits
- Allow the user to identify different objects and assign them as non – conformities / conformities.
- For non – conformities, identify the ISO clause/clauses which they are in breach of.
- Inform users of any errors in their answers, as this is a training platform that should allow users to learn from mistakes.

2. PROJECT DESCRIPTION

2.1 Description of the training platform

The training platform is designed to train users to identify non-conformities (NCs) during an internal audit, and the ISO clause(s) which they failed to comply with for the identified NC. The clauses are from the ISO standard ISO 13485:2016 — Medical devices -- Quality management systems -- Requirements for regulatory purposes [1]. Users would take the on role of an internal auditor. They must find instances of noncompliance by recognizing various objects, laboratory practices and physical attire of characters that do not comply with the ISO standards.

2.1.1 Setting

The scenario that is depicted in our training platform is based on a Laboratory setting. We created a laboratory scene using store bought assets and modified it to create a scene that resembled labs used in biomedical settings.

2.1.2 Platform

The Unity game engine was used to develop the training platform, with Unity's XR interaction toolkit [2]. It is built using WebGL, which allows it to be run on web browsers. With the use of WebXR and unity-webxr-export package [3], both desktop and Virtual Reality (VR) are supported. It is deployed on the school's Learning Management System (LMS) which is Brightspace, by packaging it into a SCORM package [4].

2.2 Workflow of the training platform

Workflow of Training Platform

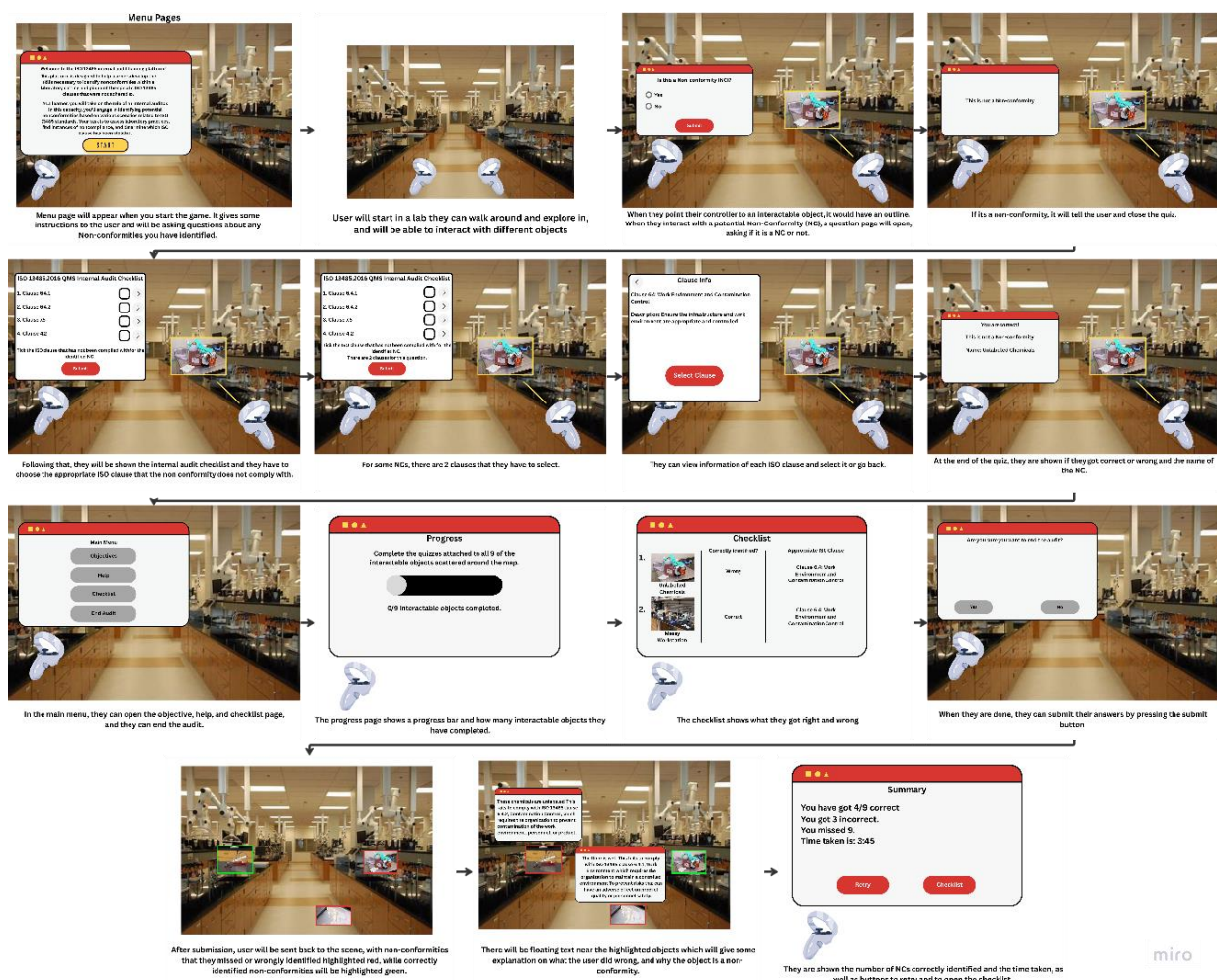


Figure 1 Flowchart of the workflow

The training platform has 3 main parts, the introduction, the internal audit and a final summary and ending.

2.2.1 Introduction

At the start, users are introduced to the premise of the game. They are informed of their role as an internal auditor and instructed to find instances of noncompliance. They can choose to proceed with the tutorial. They are brought through 3 tutorial sequences, firstly how to move around, secondly how to interact with objects and complete the quizzes, and thirdly how to navigate and use the menu. After that, the tutorial ends, and the internal audit begins.

2.2.2 Internal Audit

When the audit starts, 5 non-conformities (NCs) are randomly picked and spawned in set spawn points around the map or set up for fixed objects/special NCs. The remaining unpicked NCs get spawned as their non-NC variations. Users go around the laboratory, highlighting and selecting NCs, and completing a quiz which tests their identification of NCs and knowledge of ISO clauses. They monitor their progress using the menu's progress bar and checklist which shows completed NCs and their answers, and they can end the audit at any time, leading to the summary scene.

2.2.3 Summary and Ending

Once the audit has ended, the user is informed to walk around the laboratory to see how they did. Summary panels appear near to NCs and non-NCs, informing the user if they missed it, got it right or wrong, and the correct ISO clause for it with an explanation on why it is non-conformant to the ISO 13485 standard. After they have walked around the scene, they can view a final summary page, which displays their statistics such as how many they got correct or incorrect, and how many they missed, as well as the time taken. They can view the checklist with all of the NCs and non-NCs from the scene added with their results, and have the option to retry the internal audit with newly randomized NCs and Non-NCs.

3. PRODUCT DEVELOPMENT

3.1 Development and deployment

Firstly, we had to research what type of unity project version and all the different toolkits that we would need for our training platform to work as intended. We found that the best VR toolkit, which was the [unity-WebXR interaction toolkit](#) by De-panther and used unity version 2022.3.61f1 as it was the most compatible with the toolkit. After installing it through the package manager, we installed the XR Interaction toolkit sample scene from the samples to test if it works and switched the WebXR export version used WebXR 2020. After this, to ensure that we could upload the project into politemall properly, we had to install the SCORM API package. Since I had already converted the project to the WebGL format, all I had to do was set the compression format to disabled. After that, I went to the Scorm publishing Settings, set the version to V_2004, changed its name/file path for the zip file and built the project, allowing the ZIP file to be stored and send to our supervisor, so that she could upload it onto politemall. After creating a new project and adding the various extensions, we had to ensure that as we do our work, we are able to effectively merge and share our work with each other. To do that, we decided to use unity version control to continuously check in our various changes as changesets, so that we could merge with our teammates work. After this, we had to work on the environment.

3.2 3D Environment and Models



Figure 2 The lab environment

3.2.1 Environment

To ensure that we could complete the environment in the given time limit, we had to purchase the modern laboratory environment from the unity asset store. However, this purchased environment did not have any room that was suitable to be the laboratory that would be used in our training platform. Therefore, we had to use the assets that had come with the environment to build our own laboratory. The package had come with tables, walls, doors, microscopes and other such assets suitable to be used to build a laboratory. After this, we found that the laboratory built was too inconvenient for the user to move around and see in because the various lighting and probes inside the map distorted how the map looked through the camera and cause various performance issues due to heavy load. Hence, we had to bake the lighting produced by the light probes to stop shadows and light from affecting how the user viewed the map.

3.2.2 Non-conformity (NC) Models

Firstly, we defined a list of non-conformities (NCs) to include in the training platform. After finalizing the list of NCs and their specifications with Ms Wong Yee Shan, we began modelling the various NCs. We decided to include instances of objects which were compliant with the ISO standards, to enhance the training platform as users would have to determine if an interactable object was an NC or not, which improves their ability to recognize specific instances of non-conformance. There are 3 different types of NCs: Table NCs, Floor NCs, and Special NCs. These categories are used in the spawning logic.

Images of these NC models are in Appendix A.

There are 10 non-conformities that are in the training platform. They are:

NC Name	Description	ISO Clause
Table NCs		
Unlabelled Chemicals		Clause 7.5.8, 7.5.9 - Identification and Traceability
Messy Workstation		Clause 6.4 - Work Environment

		and Contamination Control
Food and Drink		Clause 6.4 - Work Environment and Contamination Control
Not following updated protocol (SOP)	old documents with obsolete stamp lying around	Clause 4.2.4 (d), (h) - Control of Documents
Spilled Chemical		Clause 6.4 - Work Environment and Contamination Control
Floor NCs		
Wet Floor		Clause 6.4 - Work Environment and Contamination Control
Special NCs		
Not properly disposing waste into chemical waste bins	NPC disposing waste into the incorrect type of bin	Clause 6.4.2 – Contamination Control
No/outdated/failed training record	Training record for fume hood operation is outdated	Clause 6.2 – Human Resources
Inappropriate Operation of Equipment	Fume hood sash set to incorrect height	Clause 6.2 – Human Resources
Not wearing proper PPE (in Lab setting)	NPC not wearing mask, gloves, covered shoes, lab coat, long pants	Clause 6.2 - Human Resources / Clause 6.4 - Work Environment and Contamination Control

The assets were made with a combination of 3D models that were purchased or obtained for free from the Unity Asset Store and TurboSquid (Appendix A). These assets were modified and used in combination with one another to form the models. After importing the models, a total of 12 models were used as NCs and their non-NC variants. A few models were modified using Unity ProBuilder, such as the chemical waste bin and the wet floor. NCs such as the chemical waste bin, SOP documents, and labelled chemicals used images that were created or retrieved from the internet, and the images were added to them to give them the appropriate appearances.

3.2.3 NPC Models

One of the non-conformities included in the project was the proper and improper use of personal protective equipment (PPE), for example, whether individuals in the laboratory were wearing essential safety gear such as face masks and safety goggles. To show this in the virtual environment, we incorporated human characters into the scene. Another non-conformity included was a training records indicating who is properly trained to operate specific equipment. To do this, we implemented name tags for each character using Unity's world space canvas system.

Proper and Improper Use of Personal Protective Equipment (PPE)

To show whether individuals were adhering to proper PPE protocols, such as wearing face masks, safety goggles, and lab coats, we introduced human characters into the simulation. These characters were developed using Reallusion's Character Creator 3 (CC3) with additional assets such as lab coats and accessories sourced from the Reallusion content store. After creating the characters in CC3, we imported them into iClone to assign appropriate static poses. The finalized characters were then exported to Unity using the CC/Unity Auto Setup tool to ensure compatibility and correct rendering within Unity.

This allowed us to depict scenarios of both compliance and non-compliance, making it easier to visually identify characters who were improperly equipped in the lab environment.

Representation of Training Records

To address the second non-conformity, we created a visible training record within the scene that lists the names of personnel who are certified to handle equipment such as a fume hood. In addition, each character in the environment was given a name tag using Unity's world space canvas system, with text displaying their name. This allows users to cross-reference the character's name tag with the training record, identifying whether the individual is authorized to operate the equipment.

Challenges

During the process of integrating Character Creator (CC) characters into Unity, I encountered several technical challenges related to shader compatibility and performance optimization.

Initially, when I imported CC characters directly into Unity, the characters did not display correctly. The hair appeared white instead of brown, and the scalp material created a transparent bubble over the character's head. To resolve this, I switched to using the official Character Creator to Unity importer. This tool correctly imported the character assets along with their material and shader assignments, ensuring better compatibility.

While this fixed the hair and scalp material issues, the character's skin appeared pink due to missing shaders. I discovered that this was because I had not configured the shader settings properly in the importer. Disabling the Amplify shaders allowed the character to render correctly, with proper skin appearance.

However, I then faced another issue: the game build process failed due to excessive memory usage. The initial CC characters were exported with very high-resolution textures, which significantly increased memory consumption. To address this, I re-imported the characters after reducing the texture size to 512x512 pixels before exporting them to Unity. This optimization significantly reduced memory usage and allowed the game to run smoothly without crashing.

Images

The images of the NPCs are in Appendix B.

3.3 User Interaction and Movement

3.3.1 Interacting with objects

Interacting with objects (Mouse, VR) – Y

A key component of our project is user interaction with in-game objects to identify non-conformities and answer related questions. To ensure accessibility, the application is designed to run on both desktop and virtual reality (VR) platforms, requiring dual input support for keyboard and mouse as well as VR controllers.

Desktop Implementation

For the desktop version, we implemented interaction using an Event Trigger component attached to each interactable object, paired with a Physics Raycaster component on the main camera. This setup enables point-and-click interaction with game objects using the mouse.

VR Implementation

For VR, we used the XR Interaction Toolkit provided by Unity. Each interactable object was configured using the XR Simple Interactable component, allowing users to interact with objects via VR controllers.

Interaction Types and Feedback

Highlight.cs script was developed to implement two types of interactions:

1. **Hover Highlighting** – When a user hovers over an interactable object with either a mouse (desktop) or controller (VR), the object is highlighted to indicate that it is interactable. This was achieved using the Quick Outline asset, which applies an outline to objects.
2. **Selection and Quiz Triggering** – Upon selecting an object, the system triggers the start of a quiz related to the selected item.

Enhancing NPC Visibility and Interaction

To address the challenge of inspecting NPCs (who often face lab equipment, making it difficult to view name tags or check for missing protective equipment), we added functionality that causes characters to turn toward the user upon interaction. When a user selects an NPC, a popup window appears, asking whether they wish to begin the quiz immediately or continue observing the character for more clues.

In the VR version, this interaction was extended to support touch as an alternative method of triggering character rotation and quiz prompts, using XR Direct Interactor and TouchScript.cs, allowing users to simply touch the NPC with their controller for a more immersive experience.

Challenges

One of the challenges I faced was with highlighting objects. I first experimented with a few approaches, such as changing the object's material colour or adding a coloured light that activates when the object is selected. In the end I found a free online asset called Quick Outlines, which does the job effectively.

Another issue I encountered was with the XR Simple Interactable. Initially, I used the Select Event to trigger a quiz. However, I realised that the Select Event responds to the Grab Input of the VR controller. Using the Grab Input to select things didn't feel intuitive, so we wanted to switch to using the Trigger Input instead.

After some research, I learned more about how the different events work in the XR Interaction Toolkit. The Select Event is triggered when an object is grabbed, while the Activate Event is triggered when the object is grabbed and the Trigger is pressed. By enabling the Allow Hover Activate setting I discovered on the XR Ray Interactor, we can activate an object by simply hovering over it and pressing the trigger, without needing to grab it first. This gave us a much more intuitive interaction model.

3.3.2 Movement

Cross-Platform Movement System (PC and VR)

To provide users with a seamless movement experience across both PC and VR platforms, I developed a unified script named FirstPersonMovement.cs, which handles both device detection and movement control within the same structure. The key goal was to ensure that platform-specific movement schemes would activate only when the appropriate device was

detected — creating an intuitive and immersive experience regardless of the platform.
PC Controls

When the system detects a PC environment, traditional first-person movement controls are activated:

- WASD or arrow keys for directional movement.
- Mouse or touchpad for looking around and interacting with objects.
- Click input for triggering interactions with objects in the environment.

VR Controls

For VR users, the control scheme switches dynamically:

- The left joystick on the VR controller moves the player.
- The right joystick handles smooth or snap turning.
- The trigger button on the right controller is used to interact with objects.
-

In VR mode, the camera and player rig are handled differently due to the use of XR components. I implemented custom helper functions such as `CharacterControllerHeight()` to adjust the character collider dynamically based on the user's physical height, and `RotationWithJoystick()` to manage smooth camera rotation using input from the right joystick.

While writing these functions, I used ChatGPT to help generate initial code snippets for joystick rotation and camera height adjustment. However, integrating them correctly with the XR Interaction Toolkit's device detection system—especially with Unity's evolving XR input framework—posed a real challenge. I relied heavily on examples and guidance from GitHub repositories and the official XR Toolkit documentation to resolve various integration issues.

One of the most difficult challenges was designing the system to support two completely different input paradigms—mouse/keyboard for PC and XR controller input for VR—within a single project structure.

Early versions of the script used separate components and conditional checks for VR and PC, which quickly became messy and error-prone. For example, having separate movement logic for VR and PC made the system hard to maintain and debug. Bugs would arise where movement would either break entirely or partially trigger the wrong input methods when switching platforms mid-session.

To solve this, I rewrote the core of `FirstPersonMovement.cs` to use a modular approach. Platform detection occurs at runtime using XR Toolkit's device API, and movement logic is wrapped in clearly separated methods for each platform. This allowed the project to dynamically enable only the relevant input handlers and avoid overlap.

A particularly tricky issue arose with the VR movement system—specifically with maintaining smooth rotation using the right joystick while preventing camera jitter or input lag. This was due to conflicts between Unity's input system events and XR input updates, especially when using XR rigs like XR Origin. After many iterations and testing, I implemented a frame-dependent input smoothing technique and restructured the input polling mechanism to avoid conflicts with Unity's `Update()` cycle.

3.4 Main Game Systems

3.4.1 Tutorial

At the beginning of the game, they are showed a welcome page that has options to start or skip the tutorial. In the tutorial, there are 3 stages of it.

1. Movement

The first part of the tutorial teaches the user how to move around in VR

When the tutorial begins, a thin transparent GameObject is highlighted with a red outline using the Outline.cs script from the Quick Outlines assets. This GameObject has a Box Collider component with “Is Trigger” enabled.

A Screen Space UI will appear near the top of the screen, instructing the user to walk towards the outlined area.

Once the user enters the outlined area and touches the collider, the OnTriggerEnter function in the Tutorial1.cs script is called. This disables the GameObject, signalling the end of this step and triggers the next part of the tutorial.

2. Quiz

This part of the tutorial is handled by Tutorial2Script.cs and TutorialStep.cs. Before the quiz part of the tutorial, the user is informed of various controls such as ‘Q’ to unlock the cursor and ‘M’ to open the menu. They can press ‘N’ to move to the next tutorial step as it listens for the ‘N’ key press which invokes the NextButtonPressed() function in Tutorial2Script.

When the quiz part of the tutorial begins, a random Table NC is instantiated on a set spawnpoint in SpawnTutorialObjects() in NCManager.cs. After being introduced to how the internal audit works like how many NCs will spawn, it highlights the object and tells the user to select it. The next button is hidden during this segment so the user has to follow the tutorial’s instructions to continue the tutorial. For each step, such as when OnSelect() in Highlight.cs is run when they select the object, it checks if the intro is still running and if it is, it runs the NextButtonPressed() function in Tutorial2Script.cs. Once they complete the quiz, it moves to the next part of the tutorial.

3. Menu

The tutorial part of the menu was quite simple. To ensure that the tutorial text box and button would be visible on top of the menu UI, I gave it a higher priority number than all the other UIs. I then modified TutorialStep.cs to include additional steps explaining how the menu has different pages with different functions. At the end, the menu would have a end audit button(button will call the endIntro function, disabling the gameobjects that display the tutorial scenes and start the game) that the user can press at any time to end that tutorial and move onto the game. To ensure that the checklist and objective pages would not overlap in the tutorial and actual games, I had to add listeners for endIntro function in the Intro.cs script inside InteractableObjectDisplay.cs(for objective page) and ChecklistManager.cs(for checklist page). For ChecklistManager.cs, the listener would call the ClearAllRows function, which cleared all the rows in the checklist(by deleting all the prefabs under the Content panel in the checklist). For the InteractableObjectDisplay.cs, the endIntro function would first set isdoingintro in Intro.cs to false. It would also call UpdateInteractableCount in InteractableObjectDisplay again, setting the totalInteractables and completedInteractables to 0 and recounting the number of totalInteractables.

3.4.2 NC Management and Spawning

At the beginning of the internal audit, 5 random non-conformities (NCs) are chosen from the list of 10 NCs. The information of these NCs is stored as a Unity Scriptable Object (NCObj.cs), which is a data container that stores information independently from class instances, so only one copy of the data is stored in the memory. The information includes the name, description, and Boolean flags that specify if the NC is meant to spawn on the table or the floor, and whether it has a non-NC variation. It also contains references to its prefabs, which are the 3D models of its NC and non-NC variation. Finally, it has references to its correct and false ISO clauses. ISO clauses are also stored as ScriptableObjects

(ISOClauses.cs), so it can be reused for multiple NCs. The information stored in the ISO clause ScriptableObject is its clause number, name, and description.

There are 3 main categories of NCs that are handled by the spawning system. There are Table NCs, Floor NCs and Special NCs.

Table NCs: NCs which spawn on tables and lab workbenches.

Floor NCs: NCs which spawn on the floor.

Special NCs: NCs which do not come under those 2 categories that require special handling. There are spawn points which are empty transforms that are placed around the map. They are categorized according to the categories above. These spawnpoints are stored in an array for Table, Floor and characters. The NC ScriptableObjects are stored in an array, as well as the prefab for the summary UI panel, which is instantiated with the NCs that are spawned.

NC Spawning is handled by NCManager.cs. There are 3 main stages of NC Spawning.

1. NC Randomization

At the start, a list of selected objects is created from the list of NC ScriptableObjects. Then this list is shuffled using a shuffle function using the Fisher-Yates shuffle algorithm. Then the first 5 NCs are stored in the NC Objects list, and the remaining NCs are added to a non-NC Objects list if they have a non-NC variation.

2. Spawnpoint Randomization

From the list of selected objects, the number of tables NCs and floor NCs is obtained by looping through the list and incrementing a number for both. The list of selected table and floor spawnpoints are then shuffled and then shortened to the number of table and floor NCs.

3. NC and non-NC spawning

There is a function, SpawnNCObject() in NCManager.cs that handles spawning for table and floor NCs. Special NCs such as PPE which involves characters are handled in a separate loop as they need specific spawning logic. There are 3 stages of spawning which is repeated for NC objects and Non-NC objects.

- The first stage is spawning table and floor NCs. Table NCs and floor NCs are spawned at the spawnpoints from their respective lists, and then a count for each is incremented, as the list of spawnpoints is shared for NCs and Non-NCs.
- The second stage is for characters. As there are 2 characters in the map, it randomly picks which character has the wrong PPE.

The final stage is for special NCs. They have specific handling for each different type of special NC.

A challenge which I faced was managing all the different types of NCs. Other than table and floor NCs, I needed a way to differentiate the special NCs and to develop their spawning behaviours. So I added a field in NCObj which is an enum SpecialNCType, which distinguishes the type of NC by a set list, such as PPE, Bin, TrainingRec and FumeHood. The table below shows how the Special NCs are handled in SpawnRandomObjects() in NCManager.

Name	Spawning Behaviour
PPE	Wrong PPE is randomly chosen from a list of different variations of NPCs wearing non-conforming clothing. It randomly chooses which NPC has the incorrect PPE.
Waste Bin	The waste bin (NC) and chemical waste bin (Non-NC) are fixed objects. An NPC is spawned in front of the bin that they are using. The other bin's interactability and highlight scripts are

	disabled.
Training Record	Displays 4 names in a UI. If it is an NC, the training record for the named NPC in front of the fume hood will either have a missing/outdated/failed training record.
Inappropriate Operation of Equipment (Fume Hood)	It adjusts the y position of the fume hood sash. If it is an NC, the sash would be too high.

3.4.3 Quiz System

When a user selects an interactable object, a quiz interface opens. There are 2 questions that are asked:

1. Is this a non-conformity?
2. ISO Clause selection

There are 2 main scripts that handle the quiz system, QuizManager (QuizManager.cs), a singleton, and QuizUI (QuizUI.cs). Quiz states are stored in a data class, with a reference to the NC ScriptableObject of the game object. There is an NC Reference (NCReference.cs) on each game object that contains the NC ScriptableObject as well as a isNC flag and other flags. When the NCs are spawned, this NC reference is retrieved and the NC ScriptableObject is retrieved from it, as well as the isNC flag. These get added into the quiz state which is added to a dictionary (quizStates) stored in the QuizManager singleton instance, where the key is the game object that is spawned in and the value is the quiz state.

The quiz is rendered to World Space on VR, and Screen Space Overlay for desktop. For VR, when the quiz starts, the quiz UI panel (QuizUI panels are under Laboratory gameobject) is moved to a set canvas spawnpoint (CanvasSP under NC Spawnpoint) which is a child of the main spawnpoint, and the first question is shown. QuizManager passes the current gameobject and NC reference to QuizUI, and shows the first question.

For the first question, there are 4 cases than can happen:

	NC	Non-NC
Yes	Is NC, answered Yes: Inform user that they answered correctly, Move to second question.	Is not NC, answered Yes: Informed that they answered incorrectly, end quiz.
No	Is NC, answered No: Inform user that they answered incorrectly, Move to second question.	Is not NC, answered No: Inform user that they answered correctly, end quiz

When the quiz is ended, a submit function is used from QuizManager (SubmitQuiz()). For non-NCs, the quiz is submitted and the quiz closes. For NCs, it moves to the second question.

For the second question, they are shown 4 ISO clauses. These are retrieved from the NC ScriptableObject. Both correct and incorrect clauses are stored there, and their order is randomized. If there are 2 correct clauses, the user is informed, and they must select 2 clauses to submit the quiz. When they submit the second question, the selected clause(s) is checked against the correct clause(s) in QuizManager. The quiz state is populated when the quiz is submitted, and the quiz closes. These quiz states can be retrieved from QuizManager to be displayed in the menu checklist and the summary page. Finally, a popup will show them if they had answered the quiz correctly, and close shortly after.

A challenged I faced was how to show the Quiz in VR and how to link the quiz to the spawned game objects. As we had created canvas spawn points for the summary, I was able

to reuse this to move a QuizUI panel to a set location relative to each NC's spawnpoint (QuizUI.MoveTo() and QuizManager.MoveQuizUI()). I was unsure how to associate each object with its NCObj scriptable object. I asked Github Copilot and it suggested adding the NCReference dynamically when spawning. But I realized that I could add the NCReference to each game object manually, as there are a set few prefabs that are used to spawn NCs, which is more consistent and reliable, and easier to implement.

3.4.4 Summary End Scene

Summary End Scene – Y

In our project, the user is given the option to end the auditing process at any time through an option in the in-game menu. Upon doing so, the system transitions into the Summary Phase. During the Summary Phase, all interactable game objects in the environment are highlighted using the Quick Outline asset. The colour of each outline shows the user's performance related to that object:

- **Green:** The user answered the associated quiz question correctly.
- **Red:** The user answered incorrectly.
- **White:** The object was missed—no quiz question was attempted for it.

As the user approaches any of these highlighted objects, a floating panel appears above the objects. This panel provides:

- Feedback on whether the user's answer was correct or incorrect.
- The specific ISO clause that the object violates.
- A brief explanation of why the object constitutes a non-conformity under that clause.

This interaction is designed to encourage user reflection and reinforce learning through contextual feedback.

To enable this functionality, we created a public Unity event in Exit.cs script that is invoked when the user selects "End Audit" from the menu, as well as 2 scripts, SummaryNC.cs and SummaryI.cs. This event triggers:

- The enabling of outlines on all interactable objects.
- The activation of the summary floating UI components.
- Updating text on each summary panel using data stored in a Dictionary, which contains the user's quiz results for each object.

After the user finishes reviewing all interactable, they may reopen the menu to access a Final Summary Panel. This panel includes:

- Audit performance, like time taken or how many objects they got correct or missed.
- Options to restart the game for another attempt or to quit the application.

This structured and interactive summary system supports self-paced learning and reinforces the user's understanding of ISO non-conformities through immediate, personalized feedback.

3.5 Menu System Implementation and Challenges

The menu system was designed with a main menu page containing four primary buttons, each responsible for opening its respective page: Objective, Help, Checklist, and Exit Audit.

- **Objective Button:** Opens a page displaying the total number of interactable objects (both NC and non-NC) and tracks how many the user has interacted with. A progress bar visually represents completion status. This is powered by a flag in the QuizManager script, which increments each time a quiz is completed for an object, thereby updating

the progress bar.

- **Help Button:** Provides a brief set of instructions depending on the platform:
 - **PC:** Open/close menu (press M), move (WASD or arrow keys), interact (mouse or touchpad).
 - **VR:** Open/close menu (press A), move (joysticks), interact (trigger button).
- **Checklist Button:** Opens a page showing each object's name, an image, and the correct answers for that object's quiz. This is populated using the quizStates dictionary, accessed via the ncGO key.
- **Exit Audit Button:** Triggers a confirmation page asking the user if they wish to end the audit. Confirming moves the user to a summary scene that presents correct and incorrect responses. Choosing "No" returns them to the main menu. All navigation is handled through Unity's OnClick event triggers.

Dual Platform Compatibility (PC and VR)

To ensure cross-platform compatibility, **two distinct menus** were created: one for **PC** and one for **VR**. A new script, MenuToggleUI.cs, was developed using the same device detection logic from FirstPersonMovement.cs. This script activates the correct menu based on platform detection:

- **PC:** Activates the screen-space overlay menu for easier viewing and toggles with the **M** key.
- **VR:** Activates a world-space menu anchored a fixed distance in front of the headset, toggled with the **A** button. This logic is handled in VRMenuActivationUsingButton.cs.

Since the sample XR Interaction Toolkit lacked clear documentation on input actions, I implemented a custom input system and linked it to the **Show Menu** action.

Challenges and Lessons Learned

One of the key challenges was managing menu activation for both platforms. Initially, I wrote two separate scripts to control the PC and VR menus independently. However, this led to redundant logic and conflicts in XR device detection, causing unintended activation/deactivation of UI elements.

Eventually, I merged the two scripts into a single unified controller. While this solved the core issue, the merging process was time-consuming due to overlapping logic and missed redundancies. Although I consulted ChatGPT for debugging help, I found its code suggestions didn't cleanly remove redundant logic. Instead, I relied more on the XR Interaction Toolkit documentation and open-source GitHub repositories for accurate implementation references.

This project helped me understand the intricacies of designing multi-platform UI systems in Unity, especially in handling input, canvas rendering modes, and device detection logic. It also taught me the importance of **modular design** early on to avoid messy integration later.

4. RESULTS

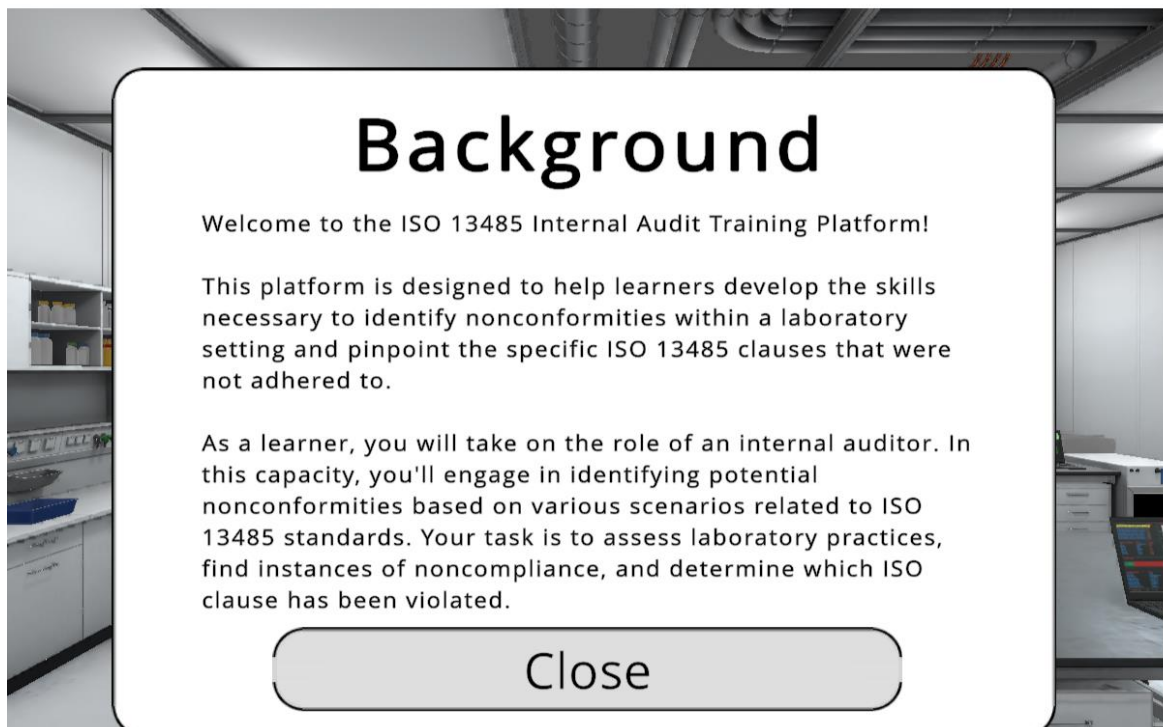
4.1 Final Training Platform (PC)

Welcome Page

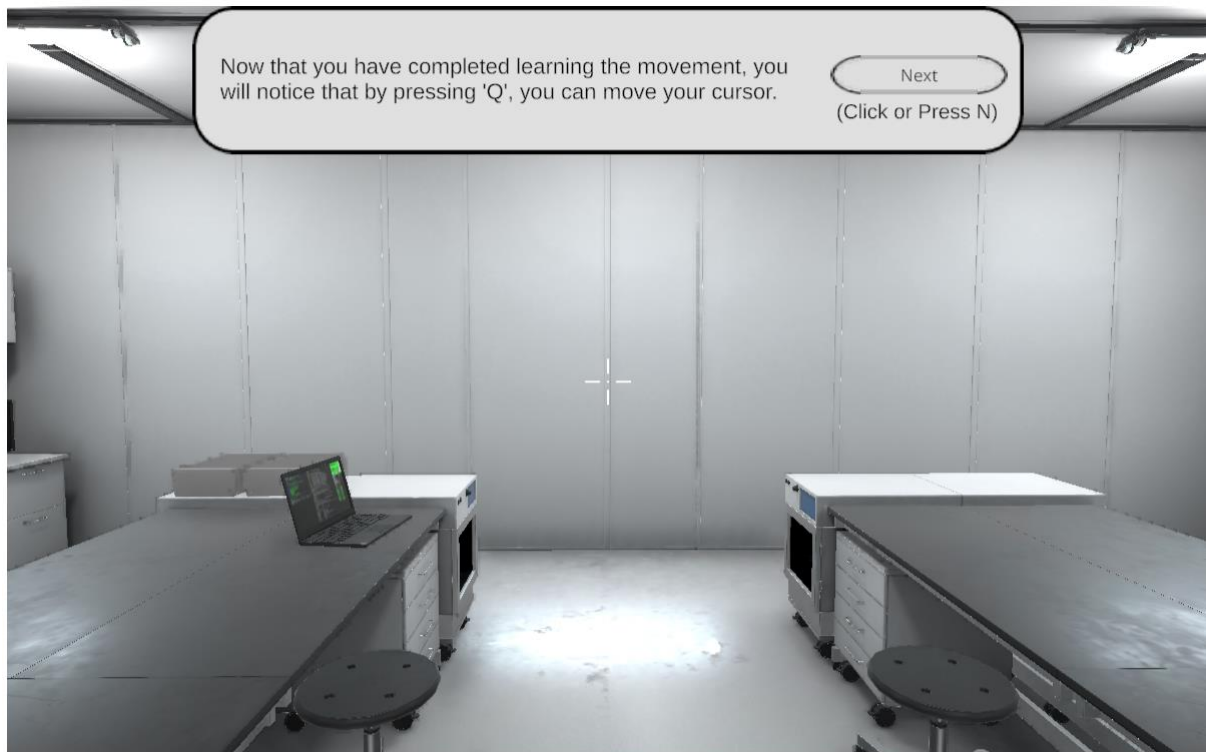
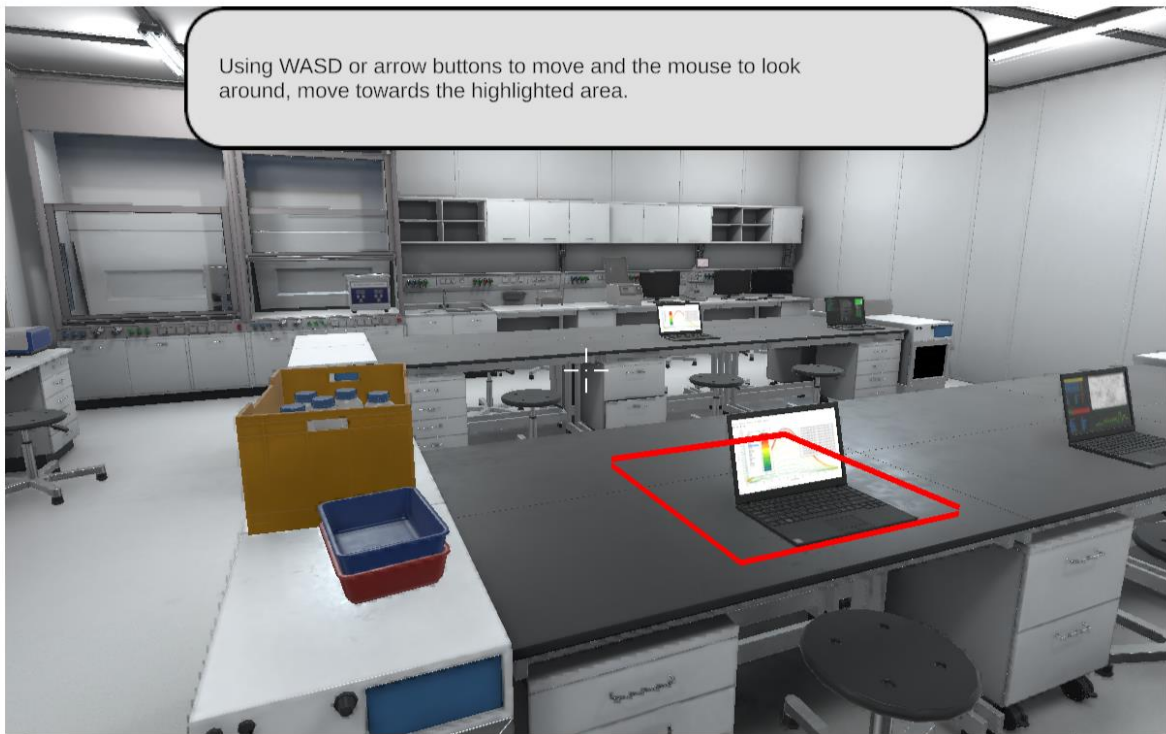


Tutorial

Background Page – shows what the training platform is used for

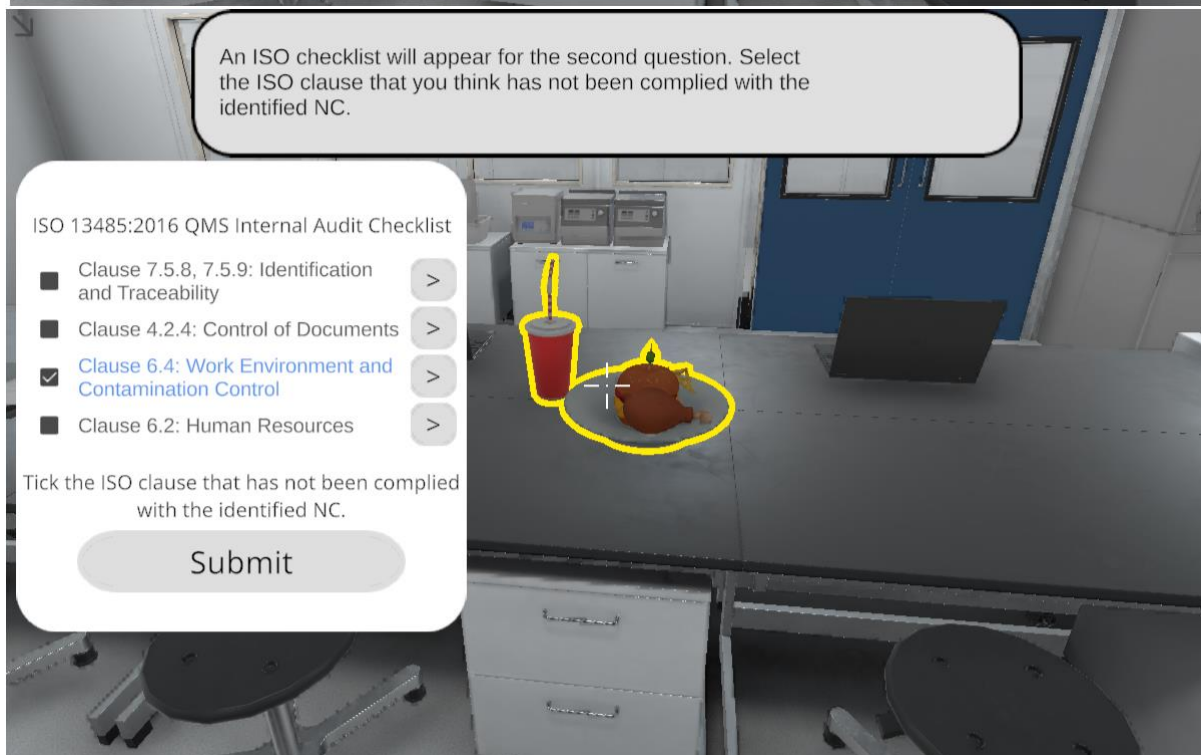
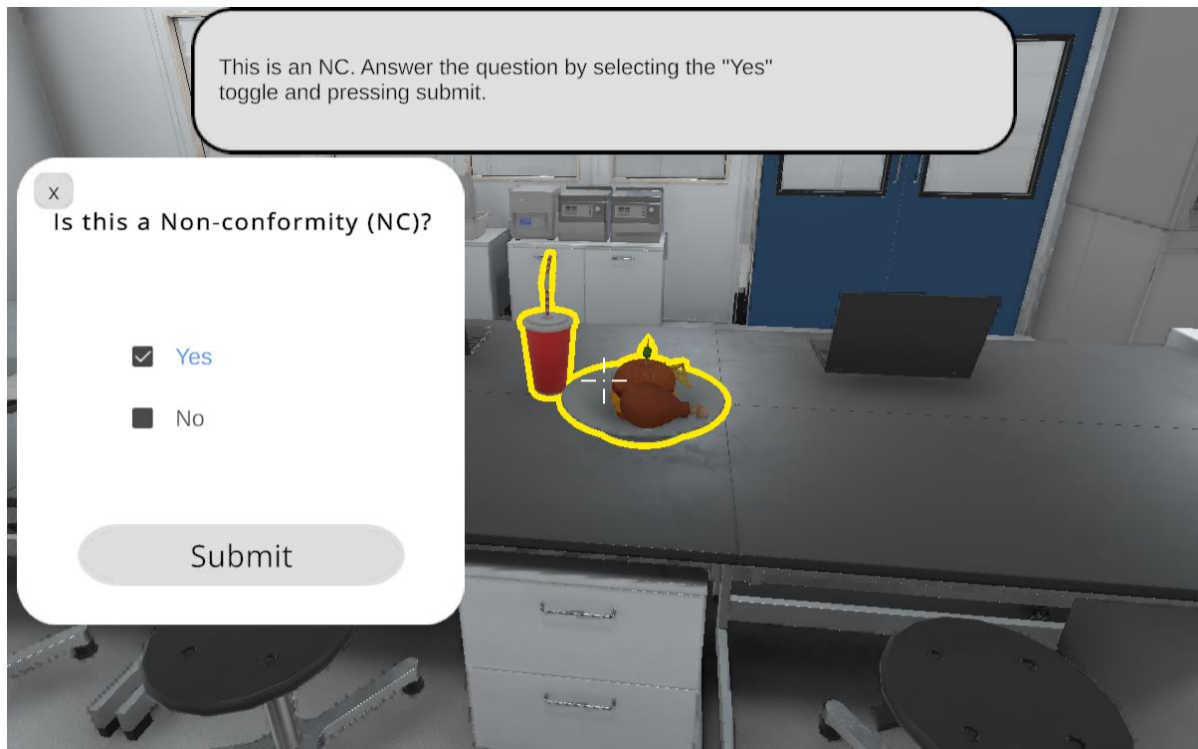


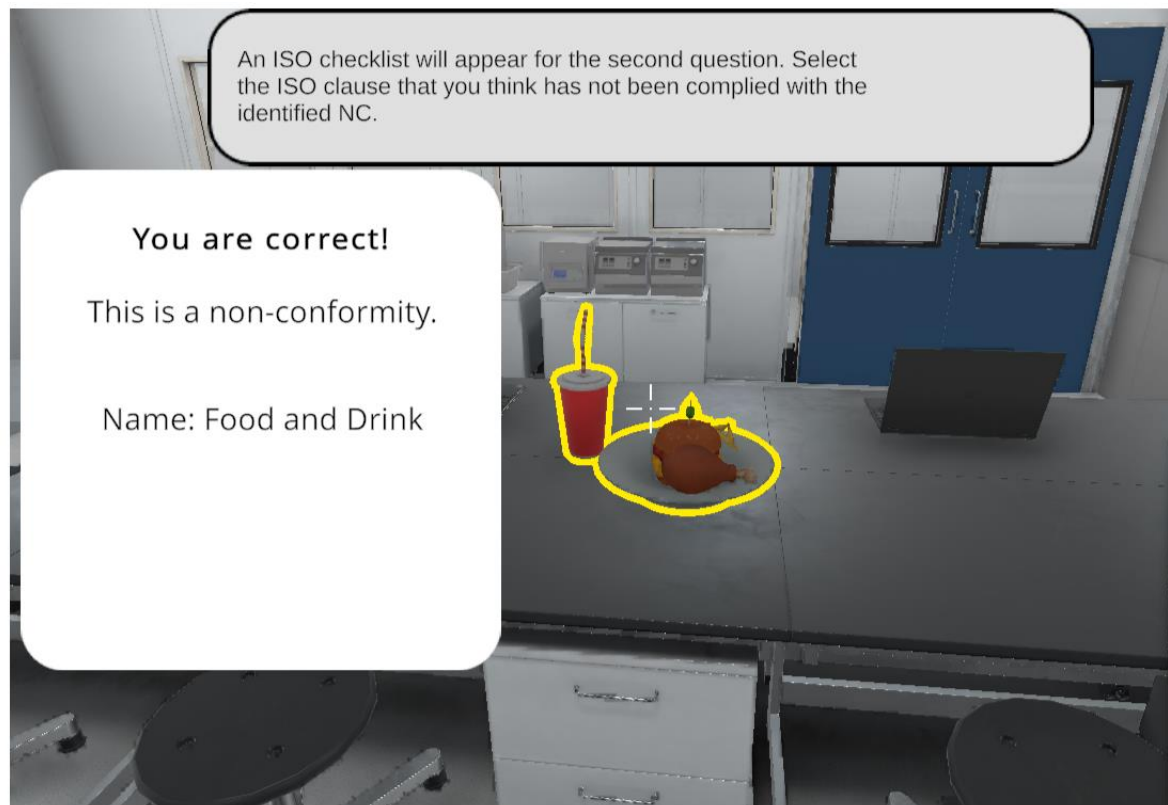
Movement tutorial – teaches user how to move around and use shortcuts on keyboard



There will be multiple instruction similar to this telling them things like pressing 'M' to activate menu and 'Q' to lock/unlock the cursor.

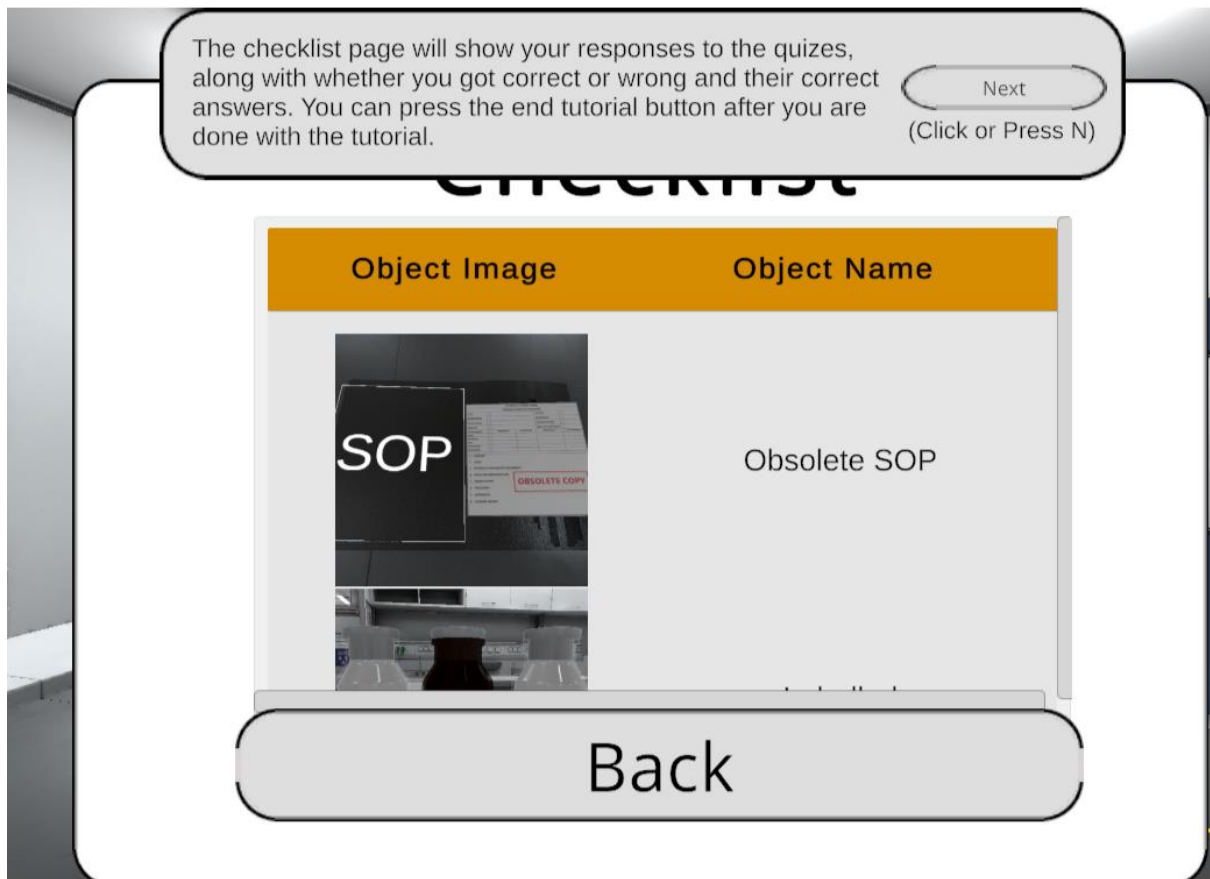
Quiz Tutorial – teaches user how to answer quizzes





Menu Tutorial – teaches user different parts of the menu





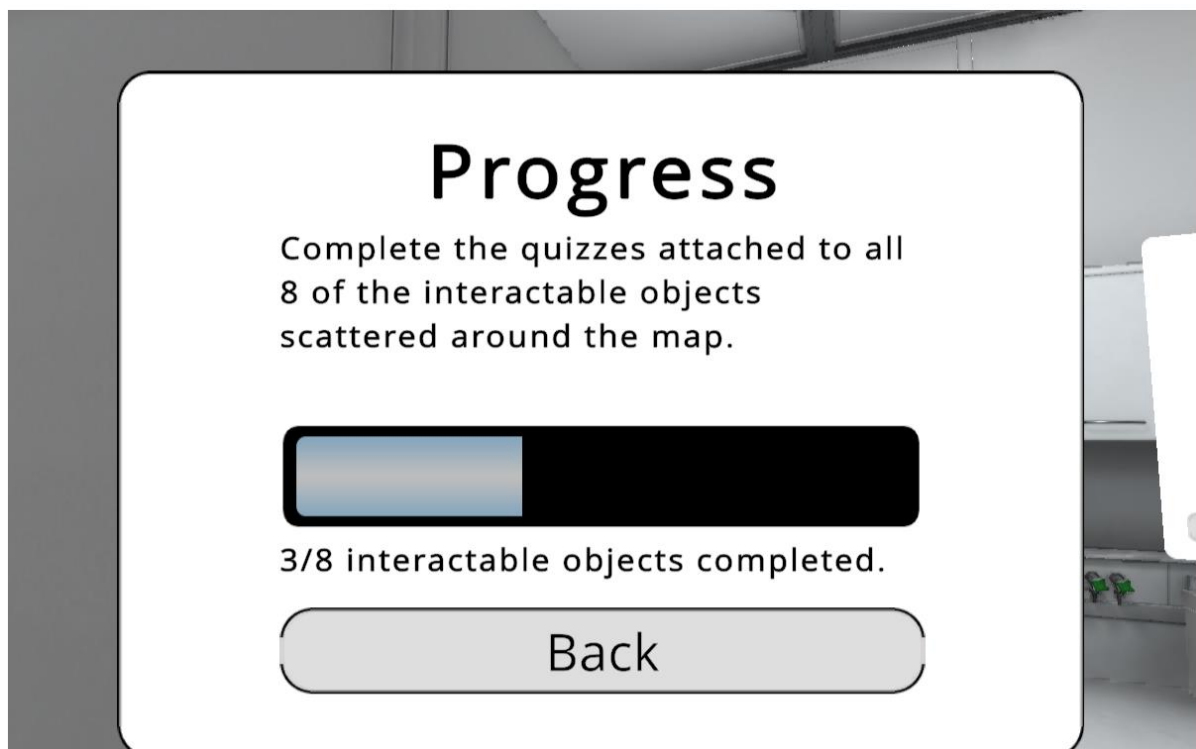
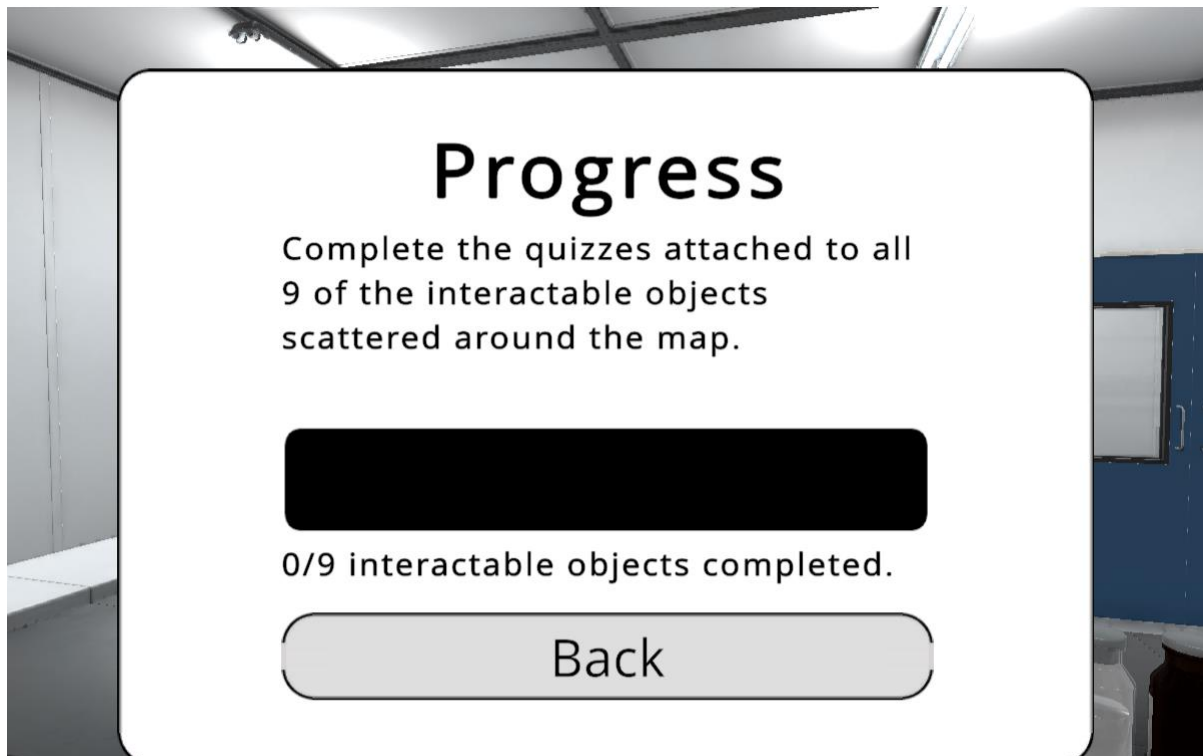
After this, you can go back to the main menu to End the Tutorial.

Game

Main Menu – all the buttons to redirect to the different menu pages

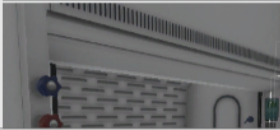


Progress Page – shows how many total interactable objects there are and how many are left



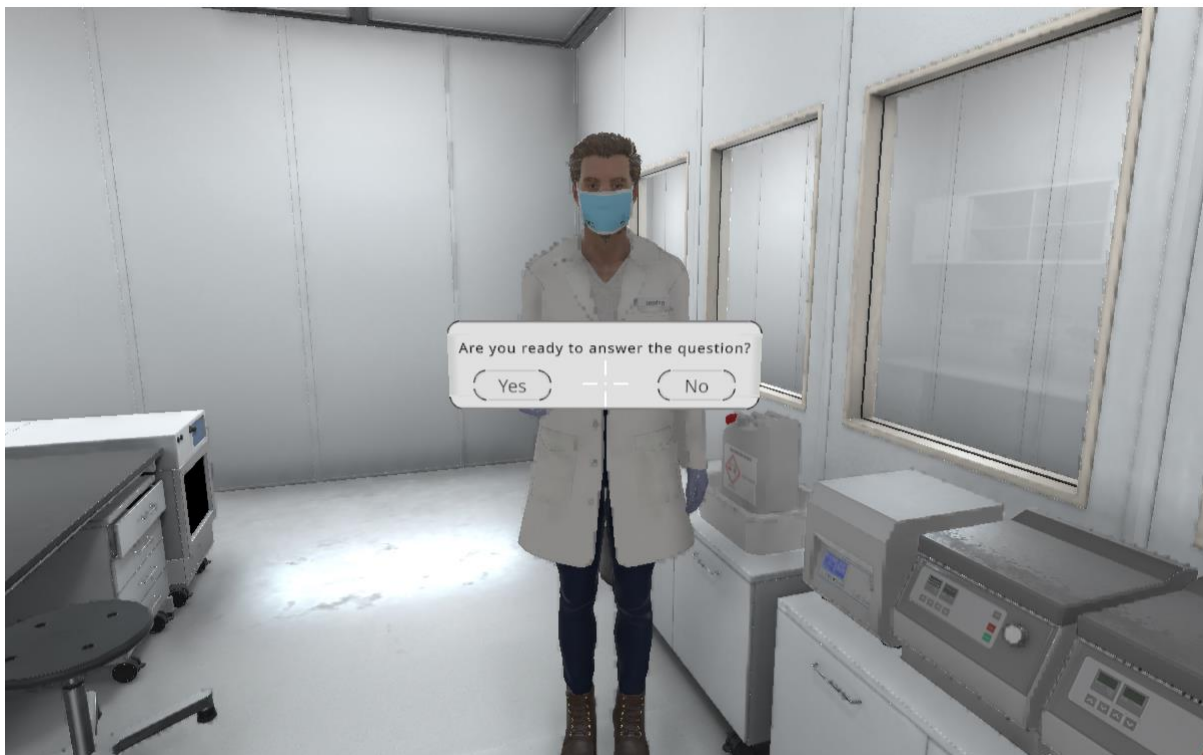
Checklist Page- displays all the details regarding the quiz(NC or non – NC, what is its correct ISO clauses, which of the questions u got correct etc..)

Checklist

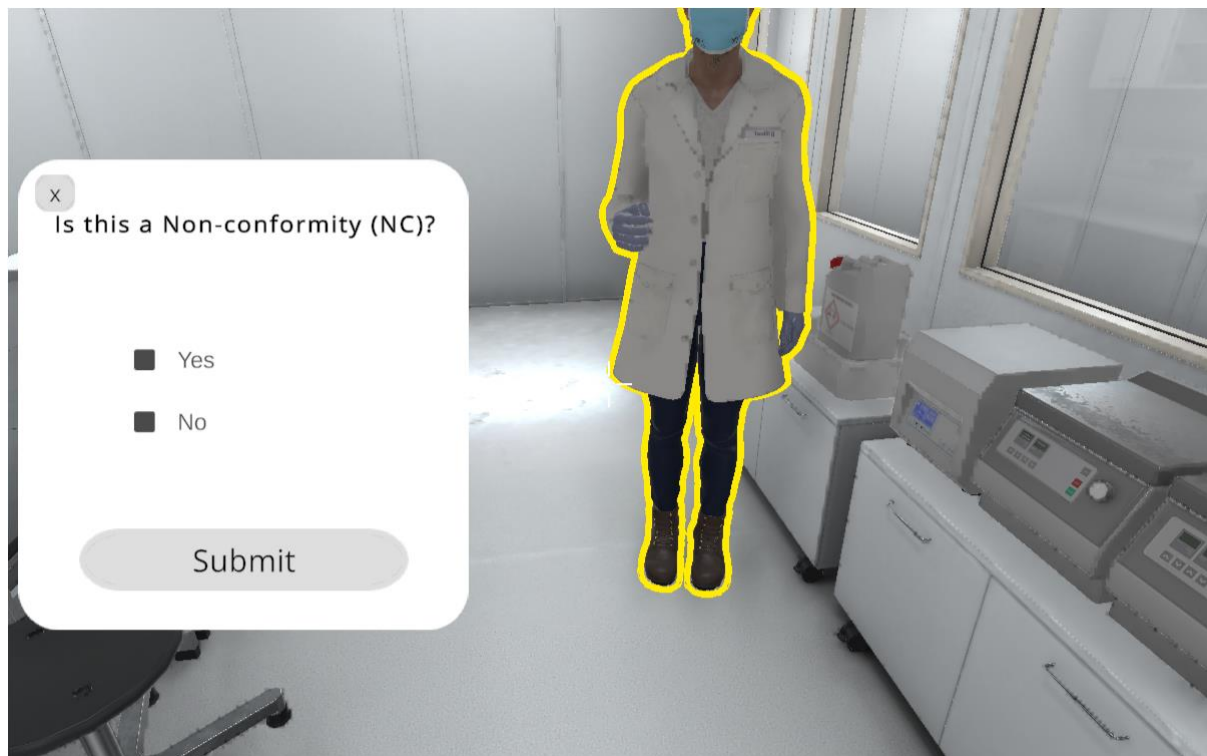
Object Image	Object Name
	Messy Workstation
	Appropriate

Back

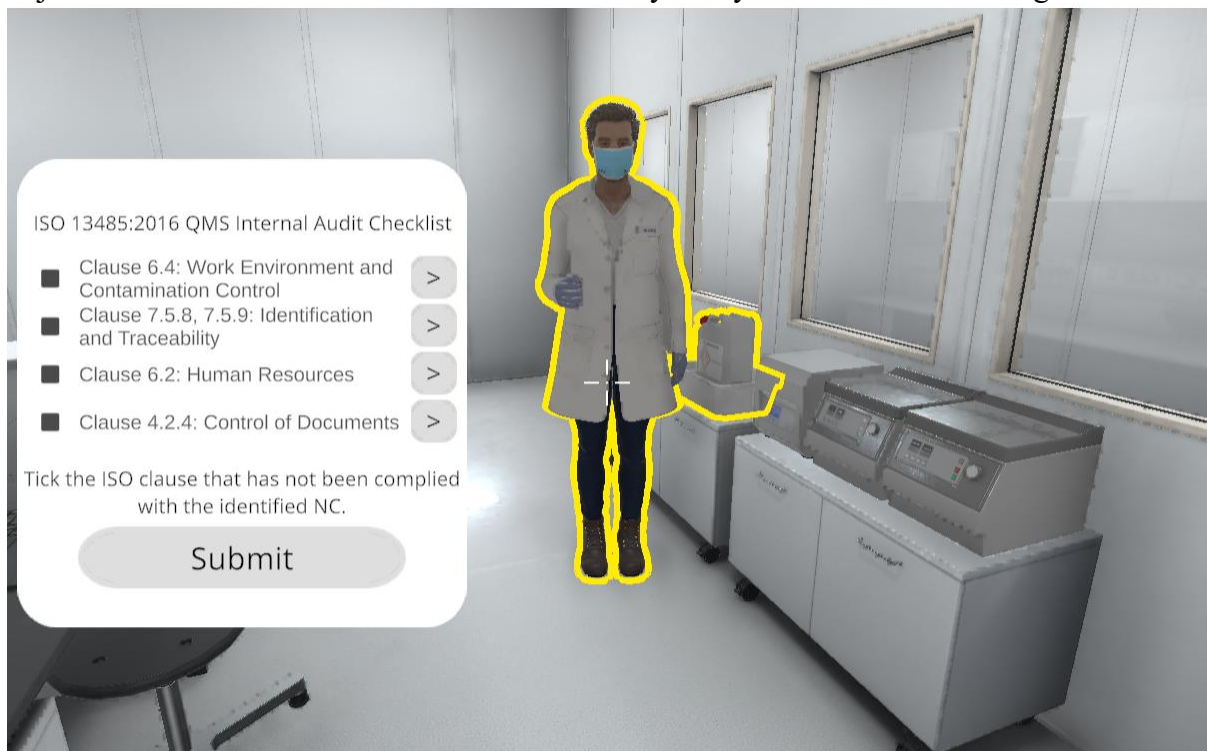
Character quizzes – When interacting with characters, characters will turn to face user and ask if they are ready to answer the quiz



All quizzes start with same question – asking if it is an NC or not



If it is an NC, it will move onto the second question, asking which ISO clause the object/character violates. Based on that it will tell you if you are correct or wrong.



When object quiz is completed, the object will have a yellow highlight



Lab design



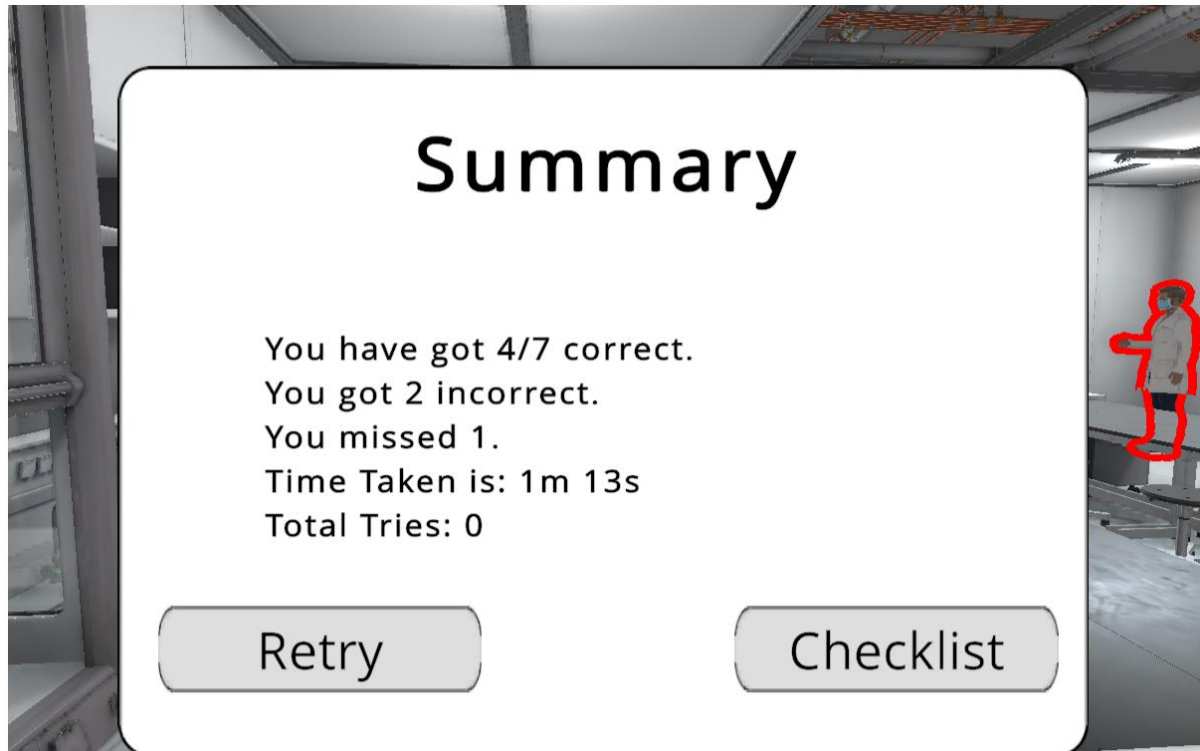
After ending the audit, the summary scene will appear, with the object being highlighted red if it was wrong overall, green if it was correct overall and white if it was unanswered.



Going closer to an object will show its summary



You can retry the game to reset everything and try again(all interactable objects spawned will be randomised) and you can open the chunklist again.



4.2 Training Platform (VR)

5. CONCLUSION

We have developed an immersive training platform for ISO 13485 internal audits which tests the user to identify different objects and assign them as non – conformities or as conformities, and to identify which ISO clause they breach if it's a non-conformities.

The training platform has been accepted by Ms Wong Yee Shan for use with actual lessons with students to train them to identify non-conformities in lab environments for ISO 13485 internal audits. With the implementation of our training platform, this would improve the effectiveness of the training with a more realistic and engaging form of training, rather than just learning by the book. It is significant as with the improved learning and practical skills, the learners will be able to more effectively identify non-conformities in real situations.

The ISO causes the non-conformities are all quite similar, if we were to continue to work on this in the future, we in include a greater variety of non-conformities that violates various different clauses in order to really test the user.

In VR, the game might be a little dizzy for users who are not used to VR. In the future we could implement features to mitigate this issue. One example is to limit field of vie when turning to reduce motion sickness.

Another area for improvement is to add more maps, rather than having the user explore the same map every time. This was not possible due to time constraints but would better teach the user to identify non-conformities in different lab environments.

Acknowledgments

The authors wish to thank Ms Oei Poh Hoon (Major Project Supervisor) for her careful guidance and patience in advising and helping us, and Ms Wong Yee Shan for her cooperation and assistance in conceptualizing and designing the training platform.

References

- [1] “ISO 13485 - Medical Devices,” ISO, <https://www.iso.org/iso-13485-medical-devices.html> (accessed Aug. 3, 2025).
- [2] “XR Interaction Toolkit: XR interaction toolkit: 2.6.3,” XR Interaction Toolkit | 2.6.3, <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@2.6/manual/index.html> (accessed Aug. 3, 2025).
- [3] De-Panther, “De-Panther/Unity-webxr-export: Develop and export webxr experiences using Unity WebGL,” GitHub, <https://github.com/De-Panther/unity-webxr-export> (accessed Aug. 3, 2025).
- [4] “SCORM API: Integration: Unity Asset Store,” Integration | Unity Asset Store, <https://assetstore.unity.com/packages/tools/integration/scorm-api-53523> (accessed Aug. 3, 2025).

Other Sources

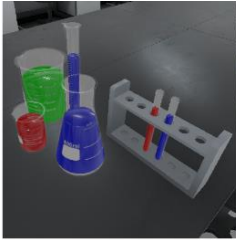
Quick Outline: <https://assetstore.unity.com/packages/tools/particles-effects/quick-outline>
CC/iC Unity Importer: https://github.com/soupdag/cc_unity_tools URP/releases/tag/1.6.3.URP10
Reallusion: <https://www.reallusion.com/contentstore/pack/character-creator/3d-cloth/male-labcoat>
<https://www.reallusion.com/contentstore/pack/character-creator/3d-cloth/mens-summer-wardrobe?addfrom=19.00086>
Chemicals: <https://assetstore.unity.com/packages/3d/props/tools/chemistry-laboratory-equipment-163610>
Food and Drink: <https://assetstore.unity.com/packages/3d/environments/foods-and-drinks-pack-208055>
Bins: <https://assetstore.unity.com/packages/3d/environments/urban/garbage-bins-crates-and-other-props-133010>
Documents: <https://assetstore.unity.com/packages/3d/props/office-props-pack2-237623>
Modern Laboratory: <https://assetstore.unity.com/packages/3d/environments/modern-laboratory-167475>
Fume Hood: <https://www.turbosquid.com/3d-models/laboratory-fume-hood-2-964235>

Appendices

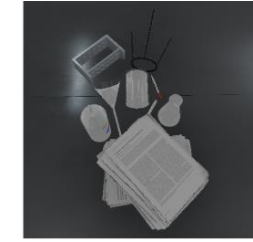
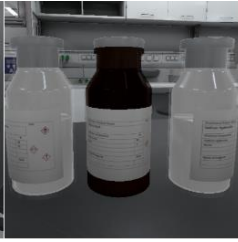
Appendix A: NC and Non-NC Models (non-NPC)

Appendix B: NPC Models

Appendix A: NC and Non-NC models



Unlabelled Chemicals



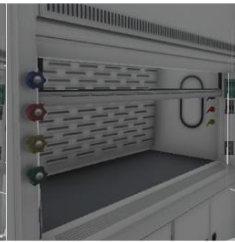
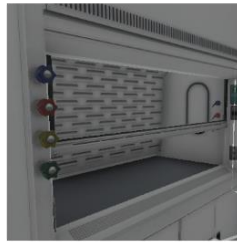
Messy Workstation



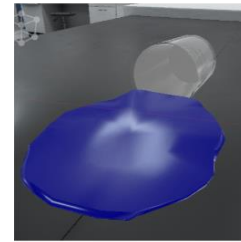
Obsolete SOP



Wet Floor



Inappropriate Operation of Equipment



Spilled Chemical



Training Record



Improper Waste Disposal



Food and Drink



Not wearing proper PPE
(In Lab setting)

miro

Appendix B: NPC Models

